

Reviewer Pack — Matroid Polytope Integrality and Circuit Complexity of MCSP

Entry c7987efbd095 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 00:51:47 UTC

Matroid Polytope Integrality and Circuit Complexity of MCSP

Entry ID: c7987efbd095 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 00:51:47 UTC

1. Conjecture

Field A (mathematical branch): MATROID_POLYTOPES **Field B** (complexity object): CIRCUIT_COMPLEXITY

Statement:

For a matroid represented by a matrix over the rationals, the integrality of its matroid polytope is polynomial-time equivalent to the circuit complexity of the minimum circuit size problem (MCSP) for certain families of functions.

Rationale (proposer’s reasoning):

The integrality of matroid polytopes relates to the structure of the matroid, which could influence the complexity of representing it as a circuit. This connection is underexplored in complexity theory.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c22ef1f1319d15b2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -matroid polytope integrality circuit complexity MCSP-polynomial-time equivalent matroid polytope integrality circuit complexity -minimum circuit size problem matroid polytope integrality polynomial-time equivalence

Top relevant hits considered: - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/2404.09152v2] A matroid polytope approach to sharp affine isoperimetric inequalities for volume decomposition functionals - [http://arxiv.org/abs/1608.04025v1] Quasi-matroidal classes of ordered simplicial complexes - [http://arxiv.org/abs/1606.00596v3] Randomized Polynomial Time Identity Testing for Noncommutative Circuits - [http://arxiv.org/abs/2204.02097v2] Simulated Annealing is a Polynomial-Time Approximation Scheme for the Minimum Spanning Tree Problem - [s2:10.1007/978-3-540-72792-7] Integer Programming and Combinatorial Optimization, 12th International IPCO Conference, Ithaca, NY, USA, June 25-27, 200 - [s2:9b121a038510ecbe2c8d51fd23a0163778979240] Computational algebraic analyses for unimodular or Lawrence-type integer programs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(b)
    augmented_matrix = [[A[i][j] for j in range(n)] + [b[i]] for i in range(n)]

    for i in range(n):
        # Find the pivot row
        max_row = i
        for k in range(i+1, n):
            if abs(augmented_matrix[k][i]) > abs(augmented_matrix[max_row][i]):
                max_row = k

        # Swap rows
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]

        # Eliminate non-pivot elements in the current column
        for j in range(i+1, n):
            factor = Fraction(augmented_matrix[j][i], augmented_matrix[i][i])
            for k in range(n + 1):
                augmented_matrix[j][k] -= factor * augmented_matrix[i][k]

    # Back-substitution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = Fraction(augmented_matrix[i][-1], augmented_matrix[i][i])
        for j in range(i-1, -1, -1):
            augmented_matrix[j][-1] -= augmented_matrix[j][i] * x[i]
```

```

    return x

def is_integer_solution(A, b):
    solution = gaussian_elimination(A, b)
    for val in solution:
        if not val.denominator == 1:
            return False
    return True

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)

    # Generate a random matrix A and vector b
    A = [[random.randint(-10, 10) for _ in range(n)] for _ in range(n)]
    b = [random.randint(-10, 10) for _ in range(n)]

    metric_value = is_integer_solution(A, b)
    conjecture_holds = True
    counterexample = ""

    if not metric_value:
        conjecture_holds = False
        counterexample = "Non-integer solution found"

    return {
        "metric_name": "Integer Solution",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 30)) # Default

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = (sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results)) ** 0.5
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Non-integer solution found\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

    'value': False, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Non-integer solution found'}
TRIAL: {'metric_name': 'Integer Solution', 'metric_value': False, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Non-integer solution found'}
TRIAL: {'metric_name': 'Integer Solution', 'metric_value': False, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Non-integer solution found'}
TRIAL: {'metric_name': 'Integer Solution', 'metric_value': False, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Non-integer solution found'}
TRIAL: {'metric_name': 'Integer Solution', 'metric_value': False, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Non-integer solution found'}
TRIAL: {'metric_name': 'Integer Solution', 'metric_value': False, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Non-integer solution found'}
TRIAL: {'metric_name': 'Integer Solution', 'metric_value': False, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Non-integer solution found'}
TRIAL: {'metric_name': 'Integer Solution', 'metric_value': False, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Non-integer solution found'}
TRIAL: {'metric_name': 'Integer Solution', 'metric_value': False, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Non-integer solution found'}
TRIAL: {'metric_name': 'Integer Solution', 'metric_value': False, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Non-integer solution found'}
TRIAL: {'metric_name': 'Integer Solution', 'metric_value': False, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Non-integer solution found'}
TRIAL: {'metric_name': 'Integer Solution', 'metric_value': False, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Non-integer solution found'}
RESULT: FALSIFIED counterexample="Non-integer solution found" first failing seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test only uses $n=1$ instances, which is insufficient to establish polynomial-time equivalence. The metric ‘Integer Solution’ appears to measure polytope integrality directly, but the conjecture links this to MCSP complexity, which isn’t addressed in the test. The single counterexample may stem from selection bias or trivial sub-cases.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test found a counterexample but only used $n=1$ instance, violating the pre-registered support condition. The metric ‘Integer Solution’ doesn’t address MCSP complexity linkage required by the conjecture. | next: Test with $\geq 80\%$ support fraction using diverse matroid families and MCSP parameterizations

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	136524
2	propose	ollama_remote	qwen3:8b	0	0	160399
3	preregistration	ollama_remote	qwen3:8b	0	0	31948
4	novelty	ollama_remote	qwen3:8b	0	0	27431
5	novelty	ollama_remote	qwen3:8b	0	0	24214
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16083
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7298
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13453
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9770
10	critic	ollama_remote	qwen3:8b	0	0	28390
11	judge	ollama_remote	qwen3:8b	0	0	21005

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 476516 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in *research/audit/c7987efbd095.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/c7987efbd095.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/c7987efbd095.tar.gz* (if generated)